Playwright Python · český cheatsheet (45 položek)

Všechny příklady používají synchronní API. `page` je karta, `field` Locator, `context` oddělená relace, `browser` běžící prohlížeč. Fixture je způsob přípravy objektů v pytestu; není to další druh stránky. Import: `from playwright.sync_api import Page, expect`. Pro regulární výrazy `import re`.

Příklady předpokládají odpovídající stránku a přihlášení. Jde o referenci syntaxe, nikoliv o 45 samostatně spustitelných testů. Kompletní scénáře najdete v testech.

# / název a syntaxe	Účel a parametry	Návrat a typický příklad	Častá chyba
01 `page` fixture	pytest předá novou kartu do parametru funkce	`Page`; `def test_form(page: Page): ...`	Ruční volání fixture nebo opětovné použití jedné stránky mezi testy
02 `context` fixture	Izolované cookies a storage jednoho testu	`BrowserContext`; `context.cookies()` vrátí seznam cookies	Záměna za browser proces; cookies mohou být citlivé
03 `browser` fixture	Sdílený proces browseru v rámci session	`Browser`; `browser.version` je řetězec verze	Vytváření vlastních contexts obchází nastavení pluginu, pokud je nepředáte
04 `page.goto(url)`	Navigace na absolutní URL či relativní cestu s base_url	`Response` nebo `None`; `page.goto('/login')`	Natvrdo port 5000 v automaticky spouštěném demu
05 `page.reload()`	Znovunačtení aktuální stránky; volitelné timeout/wait_until	`Response` nebo `None`; po úpravě `page.reload()`	Kontrolovat jen toast a neověřit trvalejší stav
06 `page.url`	Aktuální URL, vlastnost bez závorek	`str`; `print(page.url)`	V UI asserci číst jednorázovou hodnotu místo `to_have_url`
07 `get_by_role(role, name=..., exact=True)`	Hledání podle sémantické role a přístupného jména	`Locator`; `page.get_by_role('button', name='Přihlásit se')`	Jméno není vždy totéž co text potomka; více shod
08 `get_by_label(text, exact=True)`	Formulářové pole podle propojeného labelu	`Locator`; `page.get_by_label('E-mail', exact=True)`	Vybrat text labelu přes get_by_text a pokoušet se jej fill
09 `get_by_test_id(value)`	Stabilní domluvený data-testid atribut	`Locator`; `page.get_by_test_id('record-row')`	Zaměnit jej za HTML id; více řádků je očekávaných
10 `get_by_text(text, exact=True)`	Hledání podle textu, parametr může být regex; viditelnost ověří akce nebo assertion	`Locator`; `page.get_by_text('Žádné odpovídající záznamy.', exact=True)`	U ovládacích prvků přehlédnout vhodnější roli
11 `get_by_placeholder(text)`	Pole podle placeholderu	`Locator`; `page.get_by_placeholder('Hledat podle názvu…')`	Placeholder nahrazuje hodnotu, není trvalý label
12 `locator(selector)`	CSS/XPath, případně vyhledání uvnitř oblasti	`Locator`; `page.locator('body')`	Křehké dlouhé cesty závislé na layoutu
13 `locator.filter(has_text=...)`	Omezení množiny textem či vnořeným lokátorem has	`Locator`; `page.get_by_role('row').filter(has_text='Kontrola přihlášení')`	Podřetězec odpovídá dvěma řádkům; upřesněte data
14 `locator.nth(index)`	Výběr pořadí od nuly	`Locator`; `page.get_by_test_id('record-row').nth(0)`	Řešit strict mode náhodným pořadím místo významu
15 `locator.count()`	Okamžitý počet shod	`int`; `count = page.get_by_test_id('record-row').count()`	Pro čekání na UI počet použijte to_have_count
16 `locator.click()`	Kliknutí; volitelné button, modifiers, timeout	`None`; `page.get_by_role('button', name='Načíst znovu').click()`	force=True skrývá zakrytý nebo nepřipravený prvek
17 `locator.fill(value)`	Nahradí obsah editovatelného pole řetězcem	`None`; `page.get_by_label('Název požadavku').fill('Kontrola exportu')`	Číselná hodnota bez převodu na str
18 `locator.press(key)`	Klávesa nebo kombinace nad prvkem	`None`; `page.get_by_label('E-mail', exact=True).press('Tab')`	Předpokládat fokus jiného pole
19 `locator.check()`	Zajistí zaškrtnutí checkboxu/radia	`None`; `page.get_by_label('Prioritní').check()`	Kliknutí přepíná, check určuje požadovaný stav
20 `locator.uncheck()`	Zajistí nezaškrtnutý checkbox	`None`; `page.get_by_label('Prioritní').uncheck()`	Použít na prvek, který není checkbox
21 `locator.select_option(label=...)`	Nativní select podle label/value/index; lze seznam	`list[str]` zvolených value; `page.get_by_label('Stav', exact=True).select_option(label='Hotový')`	Použít na vlastní dropdown místo nativního selectu
22 `expect(locator).to_be_visible()`	Opakované ověření viditelnosti; timeout v ms	`None`; `expect(page.get_by_role('dialog')).to_be_visible()`	Čekat jen pevné sekundy
23 `expect(locator).to_be_hidden()`	Prvek není vidět nebo je odstraněn	`None`; `expect(page.get_by_text('Načítám záznamy…', exact=True)).to_be_hidden()`	Vyhodnotit před spuštěním akce a myslet si, že bylo ověřeno načítání

# / název a syntaxe	Účel a parametry	Návrat a typický příklad	Častá chyba
24 `expect(locator).to_have_text(text)`	Přesný text po normalizaci whitespace, regex nebo seznam	`None`; `expect(page.get_by_role('alert')).to_have_text('Vyplňte e-mail i heslo.')`	Neočekávaný text potomků; seznam je pro více prvků
25 `expect(locator).to_contain_text(text)`	Ověření podřetězce či regexu	`None`; `expect(row).to_contain_text('Petr')`	Příliš obecné slovo odpovídá nesprávnému řádku
26 `expect(locator).to_have_value(value)`	Hodnota pole nebo selectu, řetězec/regex	`None`; `expect(page.get_by_label('Stav', exact=True)).to_have_value('Hotový')`	Zaměnit hodnotu inputu za jeho textContent
27 `expect(page).to_have_url(url)`	Opakované ověření celé URL, řetězec/regex	`None`; `expect(page).to_have_url(re.compile(r'/dashboard$'))`	Regex bez import re; neescapované tečky
28 `expect(locator).to_have_count(number)`	Čeká na přesný počet shod	`None`; `expect(page.get_by_test_id('record-row')).to_have_count(3)`	Počítat i hlavičku tabulky přes role row
29 `expect(page).to_have_title(title)`	Čeká na dokumentový title, řetězec/regex	`None`; `expect(page).to_have_title(re.compile('Přihlášení'))`	Zaměnit HTML title za viditelný h1
30 `expect(locator).to_be_checked()`	Stav checkboxu, volitelné checked	`None`; `expect(page.get_by_label('Prioritní')).to_be_checked()`	Kontrolovat pouze atribut checked místo aktuálního stavu
31 `expect(locator).to_be_enabled()`	Prvek lze ovládat	`None`; `expect(page.get_by_role('button', name='Uložit záznam')).to_be_enabled()`	Připravenost tlačítka sama neprokazuje uložení
32 `page.screenshot(path=..., full_page=True)`	PNG/JPEG z browser obsahu; path, type, clip	`bytes`; `page.screenshot(path='reports/portal.png', full_page=True)`	Zachycení citlivých dat; není to desktop recording
33 `context.tracing.start(screenshots=True, snapshots=True)`	Zahájí detailní tracing, volitelné sources	`None`; následně `context.tracing.stop(path='reports/manual-trace.zip')`	Spustit vedle již aktivního tracingu pluginu; ruční ukázku používat s --tracing=off
34 `context.tracing.stop(path=...)`	Dokončí trace ZIP	`None`; `context.tracing.stop(path='reports/manual-trace.zip')`	Zapomenout tracing nejprve zapnout; adresář musí existovat
35 `page.expect_response(predicate)`	Context manager čekající na síťovou odpověď	`EventContextManager[Response]`; `with page.expect_response(lambda r: '/api/records?' in r.url) as info:`	Očekávání registrovat až po akci; tělo with musí vyvolat požadavek
36 `page.route(url, handler)`	Mock nebo úprava požadavků; pattern/regex/predicate	`None`; `page.route('**/api/records?*', lambda route: route.fulfill(status=503, json={'error': 'Výuková chyba'}))`	Mockem nahradit vše a tvrdit, že prošel skutečný backend
37 `page.request.get(url)`	API požadavek sdílející cookies stránky; params/headers	`APIResponse`; `response = page.request.get('/api/records')`	Nejdřív se nepřihlásit; request fixture je naopak samostatný API context
38 `page.request.post(url, data=..., headers=...)`	Měnící API požadavek; dict se serializuje na JSON	`APIResponse`; `page.request.post('/api/reset', data={}, headers={'X-CSRF-Token': token})`	Token je třeba načíst ze své session, ne použít cizí nebo vynechat
39 `page.once('dialog', handler)`	Jednorázová reakce na nativní alert/confirm	`None`; `page.once('dialog', lambda dialog: dialog.accept())`	Registrovat až po kliknutí; HTML dialog není native Dialog
40 `page.pause()`	Pozastaví headed běh pro Inspector	`None`; v dočasné ladicí kopii `page.pause()`	Nechat jej v regresní sadě a zablokovat CI
41 `@pytest.fixture`	Registruje přípravu, parametry scope/autouse/params	Dekorátor vrací fixture definici; `@pytest.fixture` nad funkcí `def sample_title(): return 'Syntetický úkol'`	Fixture volat jako obyčejnou funkci místo parametru testu
42 `@pytest.mark.parametrize(names, values)`	Spustí funkci pro každou kombinaci daných dat	Dekorátor; `@pytest.mark.parametrize('query', ['Export', 'Oprava'])`	Hodnota názvu parametru neodpovídá argumentu funkce; tajemství v ID
43 `python -m pytest -m smoke`	CLI výběr markeru; -k filtr názvu, -v detaily	Exit code 0 při úspěchu, 1 test failure, jiné kódy jiné chyby	Z úspěchu malé vybrané části odvozovat zelenou celou sadu
44 `python -m playwright codegen --target=python-pytest URL`	Skutečný browser a Inspector pro záznam	Proces s návratovým kódem; `... http://127.0.0.1:5000/login`	Zapomenout spustit ruční demo nebo upravit absolutní URL
45 `python -m playwright show-trace PATH`	Lokální prohlížení trace, PATH je ZIP	Proces s Trace Viewerem; `... reports/failure-artifacts/.../trace.zip`	Posílat firemní trace na cizí službu nebo hledat trace po zeleném retain-on-failure běhu

V CLI nahraďte `python` projektovým `.\.venv\Scripts\python.exe`. Timeouty API jsou v milisekundách. Návrat `None` znamená, že metoda nevrací užitečnou hodnotu; případná chyba vyvolá výjimku a test ji nesmí tiše spolknout.

Úplné síťové ověření

```
 with page.expect_response(lambda r: "/api/records?" in r.url) as info:
    page.get_by_role("button", name="Načíst znovu", exact=True).click()
assert info.value.ok
expect(page.get_by_test_id("record-row")).to_have_count(3)
```

Kontrolujeme síťový výsledek i uživatelské rozhraní. Samotný status 200 nedokazuje, že UI data vykreslilo správně. Nepřidáváme time.sleep.

Oficiální reference: lokátory, assertions, pytest plugin.